

Inheritance – Types of inheritance, using super keyword, Member access rules, Preventing inheritance- using final.

Polymorphism - Dynamic binding, method overriding, abstract class and methods.

Sample Program:

7. Method overriding CO1

All the banks operating in India are controlled by RBI. (e.g. minim interest rate, minimum balance allowed, maximum withdrawal limit etc) which all banks must follow. For example, suppose RBI has set minimum interest rate applicable to a saving bank account to be 4% annually. however, banks are free to use 4% interest rate or to set any rates above it.

Write a JAVA program to implement bank functionality in the above scenario and demonstrate the member variables and functions in each class.

Testcase1:

Enter the Bank name to find the rate of Interest : RBI

RBI rate of interest is : 4%

Testcase2:

Enter the Bank name to find the rate of Interest : SBI

RBI rate of interest is : 7%

dynamic polymorphism concept. Note: Create few classes namely Customer, Account, RBI(Base Class) and few derived classes (SBI, ICICI, PNB etc). Assume and implement required

BankScenario.java

This program demonstrates **inheritance and runtime polymorphism** in Java using a banking theme. There's a parent class RBI (Reserve Bank of India) with a method `getRateOfInterest()`. Three subclasses — SBI, CanadaBank, and APGVB — **extend** RBI and **override** that method to give their own interest rate. Based on user input, the program creates the matching bank object and calls `getRateOfInterest()` — Java automatically runs the correct overridden version depending on the actual object type (this is polymorphism). CanadaBank also has an extra method (`canadaBankService()`) not present in the parent, accessed using `instanceof` and type-casting. Separately, Account and Customer classes model basic customer/account data using encapsulation (private fields with getter methods).

2. Step-wise Execution

1. Scanner object is created to read input.
2. User enters a bank name (RBI, SBI, CanadaBank, or APGVB).

3. Input is trimmed and converted to lowercase for safe comparison.
4. A switch statement creates the matching object, stored in a reference variable of the PARENT type: RBI bank.
 - "rbi" -> bank = new RBI()
 - "sbi" -> bank = new SBI()
 - "canadabank" -> bank = new CanadaBank()
 - "apgvb" -> bank = new APGVB()
 - anything else -> "Invalid bank name." printed, program exits.
5. bank.getRateOfInterest() is called.
 - > Java checks the ACTUAL object type at runtime and runs that class's overridden version (dynamic method dispatch).
6. IF the object is actually a CanadaBank (checked using instanceof):
 - > it is cast to CanadaBank
 - > canadaBankService() is called (extra method, not in RBI)
7. User enters customer name and password.
8. A Customer object is created with these values.
9. An Account object is created with a fixed account number and balance.
10. Customer name, account number, and balance are printed (using their getter methods).
11. Scanner is closed.

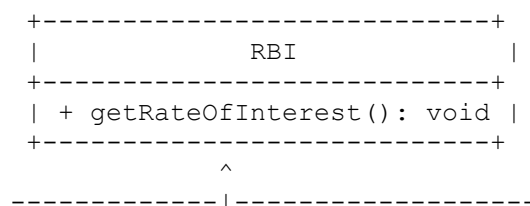
3. Algorithm

START

1. Create Scanner object
2. Read bank name from user, normalize to lowercase
3. SWITCH on bank name:
 - CASE "rbi": bank = new RBI()
 - CASE "sbi": bank = new SBI()
 - CASE "canadabank": bank = new CanadaBank()
 - CASE "apgvb": bank = new APGVB()
 - DEFAULT: print "Invalid bank name.", close Scanner, END
4. Call bank.getRateOfInterest() // runs overridden version
5. IF bank instanceof CanadaBank:
 - a. Cast bank to CanadaBank
 - b. Call canadaBankService()
6. Read customer name and password from user
7. Create Customer object with name and password
8. Create Account object with fixed account number and balance
9. Print customer name (via getName())
10. Print account number (via getAccountNumber())
11. Print balance (via getBalance())
12. Close Scanner

STOP

4. UML Class Diagram



+-----+		+-----+		+-----+	
	SBI		CanadaBank		APGVB
+-----+		+-----+		+-----+	
	+ getRateOf-		+ getRateOf-		+ getRateOf-
	Interest():		Interest():		Interest():
	void		void		void
	(overridden)		void		(overridden)
			(overridden)		
			+ canadaBank-		
			Service():		
			void		
+-----+		+-----+		+-----+	

+-----+		+-----+	
	Account		Customer
+-----+		+-----+	
	- accountNumber : String		- name : String
	- balance : double		- password : String
+-----+		+-----+	
	+ Account(String, double)		+ Customer(String, String)
	+ getAccountNumber() : String		+ getName() : String
	+ getBalance() : double		+ getPassword() : String
+-----+		+-----+	

```
+-----+
|           BankScenario           |
+-----+
| + main(String[] args) : void      |
+-----+
| creates/uses                      |
| v                                  |
Scanner, RBI (and subclasses), Account, Customer
```

Key relationships shown:

- SBI, CanadaBank, APGVB **inherit (extends)** from RBI — shown by the hollow triangle arrow (^) pointing up to the parent.
- getRateOfInterest() is **overridden** in each subclass — this enables polymorphism.
- CanadaBank has an **extra method** (canadaBankService()) accessed only after an instanceof check and downcast.
- Account and Customer are independent classes used (not inherited) by BankScenario, following **encapsulation** (private fields + public getters).

Source Code:

```
package inheritance;

import java.util.Scanner;
```

```
class RBI {
    void getRateOfInterest() {
        System.out.println("Rate of interest: 4%");
    }
}

class SBI extends RBI {
    @Override
    void getRateOfInterest() {
        System.out.println("Rate of interest: 6%");
    }
}

class CanadaBank extends RBI {
    @Override
    void getRateOfInterest() {
        System.out.println("Rate of interest: 7%");
    }

    void canadaBankService() {
        System.out.println("CanadaBank service: International
money transfer available.");
    }
}

class APGVB extends RBI {
    @Override
    void getRateOfInterest() {
        System.out.println("Rate of interest: 5%");
    }
}

class Account {
    private String accountNumber;
```

```
private double balance;

public Account(String accountNumber, double balance) {
    this.accountNumber = accountNumber;
    this.balance = balance;
}

String getAccountNumber() {
    return accountNumber;
}

double getBalance() {
    return balance;
}
}

class Customer {
    private String name;
    private String password;

    public Customer(String name, String password) {
        this.name = name;
        this.password = password;
    }

    String getName() {
        return name;
    }

    String getPassword() {
        return password;
    }
}

public class BankScenario {
```

```

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter bank name (RBI, SBI,
CanadaBank, or APGVB): ");
        String bankName =
scanner.nextLine().trim().toLowerCase();

        RBI bank;
        switch (bankName) {
            case "rbi":
                bank = new RBI();
                break;
            case "sbi":
                bank = new SBI();
                break;
            case "canadabank":
                bank = new CanadaBank();
                break;
            case "apgvb":
                bank = new APGVB();
                break;
            default:
                System.out.println("Invalid bank name.");
                scanner.close();
                return;
        }

        bank.getRateOfInterest();

        if (bank instanceof CanadaBank) {
            CanadaBank cb = (CanadaBank) bank;
            cb.canadaBankService();
        }

        System.out.print("Enter customer name: ");
        String customerName = scanner.nextLine();
    }
}

```

```

        System.out.print("Enter customer password: ");
        String password = scanner.nextLine();

        Customer customer = new Customer(customerName,
password);

        Account account = new Account("ACC1002342361",
25000765321986754300.00);

        System.out.println("Customer: " + customer.getName());
        System.out.println("Account number: " +
account.getAccountNumber());
        System.out.println("Balance: " + account.getBalance());

        scanner.close();
    }
}

```

Output:

```

PS C:\Users\Admin\OneDrive\Documents\basicjava>
PS C:\Users\Admin\OneDrive\Documents\basicjava> cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCo
deDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.BankScenario'
Enter bank name (RBI, SBI, CanadaBank, or APGB): canadabank
Rate of interest: 7%
CanadaBank service: International money transfer available.
Enter customer name: doreamon
Enter customer password: 98876552
Customer: doreamon
Account number: ACC1002342361
Balance: 2.5000765321986753E19
PS C:\Users\Admin\OneDrive\Documents\basicjava> cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCo
deDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.BankScenario'
Enter bank name (RBI, SBI, CanadaBank, or APGB): sbi
Rate of interest: 6%
Enter customer name: Avengers
Enter customer password: 23525560
Customer: Avengers
Account number: ACC1002342361
Balance: 2.5000765321986753E19
PS C:\Users\Admin\OneDrive\Documents\basicjava> cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCo
deDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.BankScenario'
Enter bank name (RBI, SBI, CanadaBank, or APGB): apgbv
Rate of interest: 5%
Enter customer name: pokemon
Enter customer password: 2223425558
Customer: pokemon
Account number: ACC1002342361
Balance: 2.5000765321986753E19
PS C:\Users\Admin\OneDrive\Documents\basicjava> cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCo
deDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.BankScenario'
Enter bank name (RBI, SBI, CanadaBank, or APGB): rbi
Rate of interest: 4%
Enter customer name: ben10
Enter customer password: 2353570554
Customer: ben10
Account number: ACC1002342361
Balance: 2.5000765321986753E19

```

